CPU vs GPU
Sequential vs Parallel

Lorenzo Pecorari - 1885161

New formulation: state aggregation

- Independent Q-Learning with Partial Observability (other agents' states aggregated)
- $s_t^i = [b_t^i, q_t^i, \tau_t^i] \rightarrow s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i]$ (9 different variables)

  where:

  - mb_t^i = argmin $\{b_t^j\}_{j \neq i}$, min battery level of other agents
  - Mb_t^i = argmax $\{b_t^j\}_{j \neq i}$, max battery level of other agents
  - mq_t^i = argmin $\{q_t^i\}_{j \neq i}$, min backlog status of other agents
  - Mq_t^i = argmax $\{q_t^i\}_{j \neq i}$, max backlog status of other agents
  - $\mu b_t^i = \frac{1}{n-1} \sum_{j \neq i}^{n} b_t^j$, average battery level of other agents
  - $\mu q_t^i = \frac{1}{n-1} \sum_{j \neq i}^{n} q_t^j$, average backlog status of other agents
- Same action space as previous formulation: $a_t = [a_t^0, a_t^1]$, where $a_t^i = [f_t^i, x_t^i, g_t^i, h_t^i]$

Setup

- 2, 3, 4 and 5 nodes, all of them as agents
- Battery capacities: 25Wh, 37Wh, 50Wh, 67Wh, 100Wh
- PV surfaces: 1.0 m², 0.85 m², 0.75 m², 0.65 m², 0.5 m² (efficiency = 20%)
- Dataset from Solcast, year 2024 (172th and 355th days mostly used for tests)
- Observation space: $s_t = [s_t^i]_{i \in [0, n]}$, where $s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i]$
- Action space: $a_t = [a_t^0, a_t^1]$, where $a_t^i = [f_t^i, x_t^i, g_t^i, h_t^i]$
- Each agent can see local state and aggregated statistics of others but not their actions
- Same environment and same reward function for both implementations
- DQN algorithm for performing learning

Extract of training code

```
for agent_id in range(0, self.num_agents):
                    self.models[agent_id]._current_progress_remaining = progress_remaining
                    if(np.random.random() < self.eps):
                        action = self.models[agent_id].action_space.sample()
                    else:
                        action, _ = self.models[agent_id].predict(obs[agent_id], deterministic=False)

                    actions_encoded[agent_id] = action
                    actions[agent_id] = self.decode(action)
```

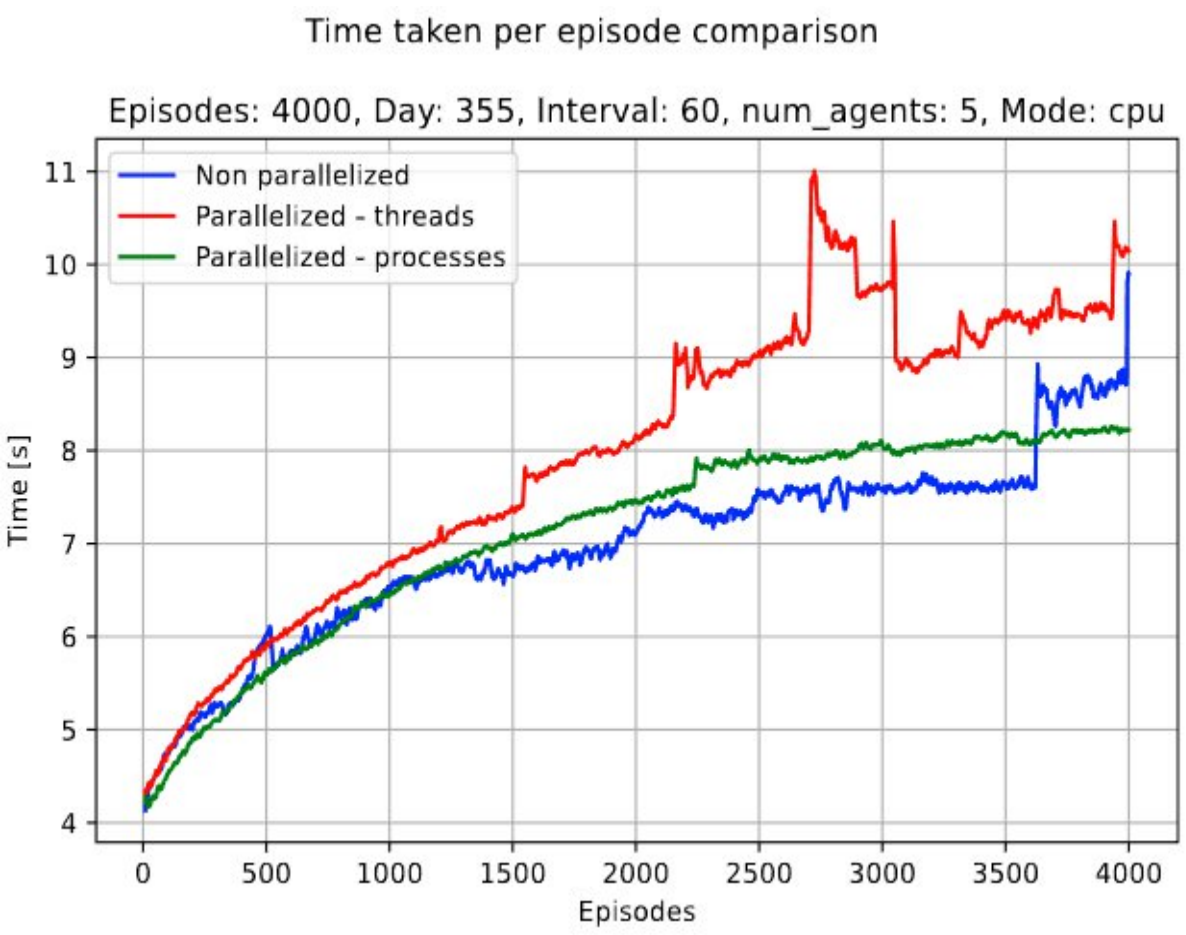
For every agent, action is chosen using ε-greedy strategy

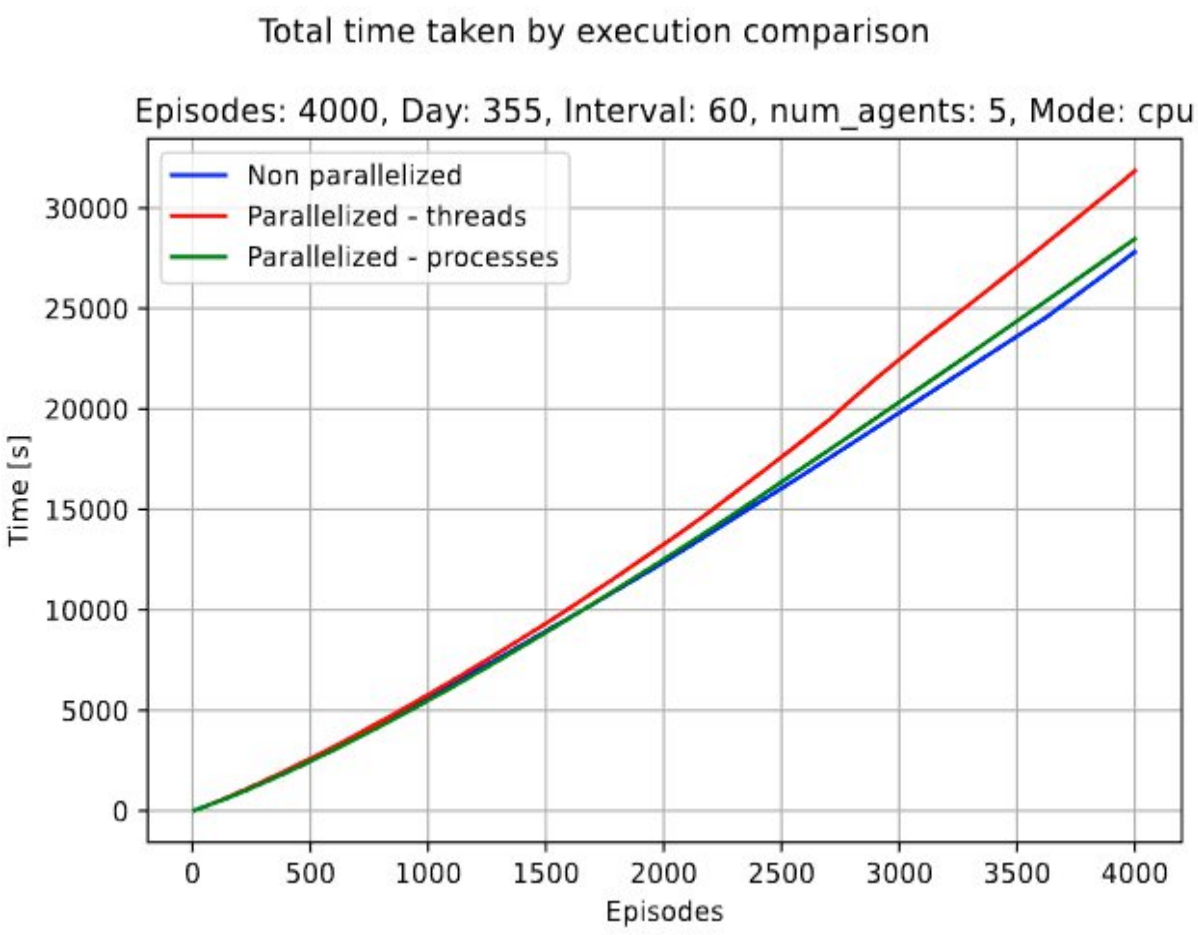
Parallelization for action selection

- Library: *concurrent.futures*
- *ThreadPoolExecutor* with num_agents workers or *ProcessPoolExecutor* with cpu_cores workers
- Differentiation between use of processes and threads
- Limitations due to Global Interpreter Lock of Python

```
self.executor = ThreadPoolExecutor(max_workers= self._max_workers)
…
futures = [
        self.executor.submit(self.get_action_single_thread, agent_id, observations[agent_id])
        for agent_id in range(self.num_agents)
        ]
for future in futures:
        agent_id, action_encoded = future.result()
        actions_encoded[agent_id] = action_encoded
        actions[agent_id] = self.decode(action_encoded)
…
```

Time comparison

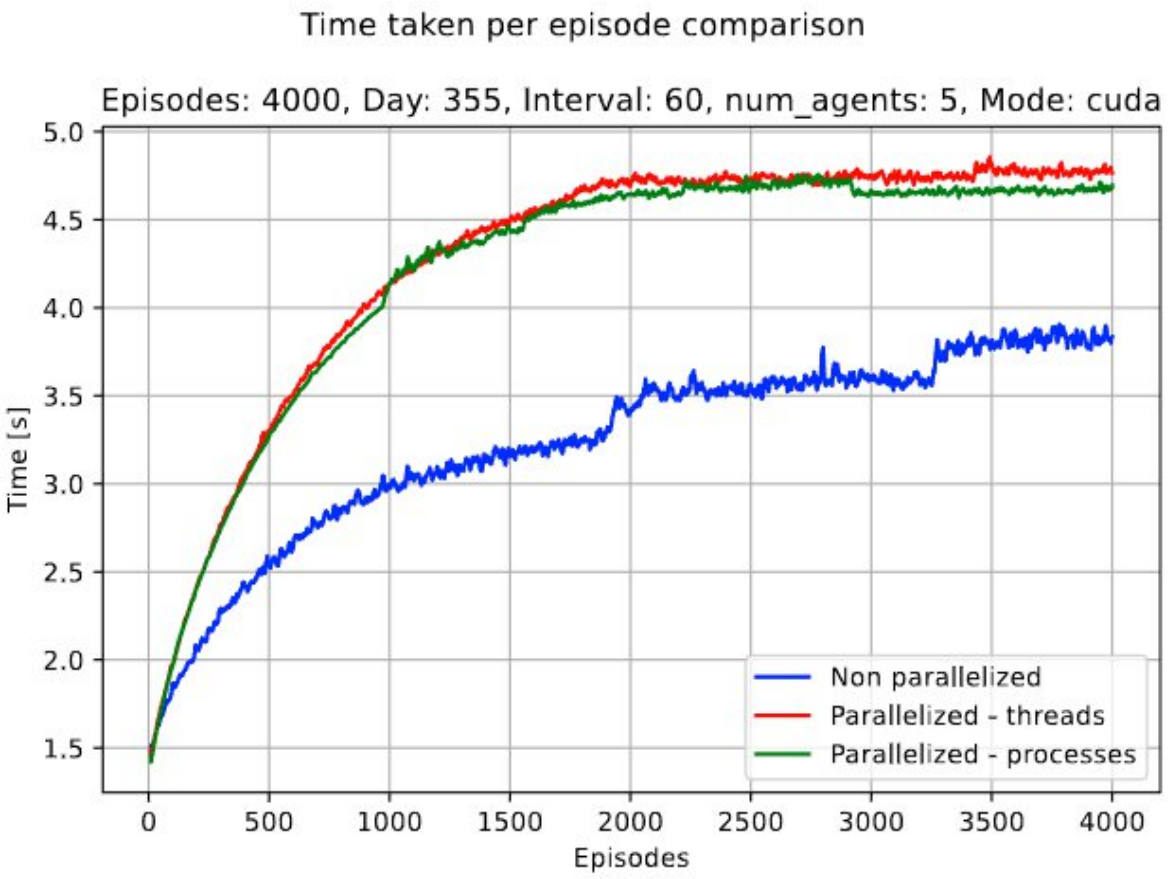


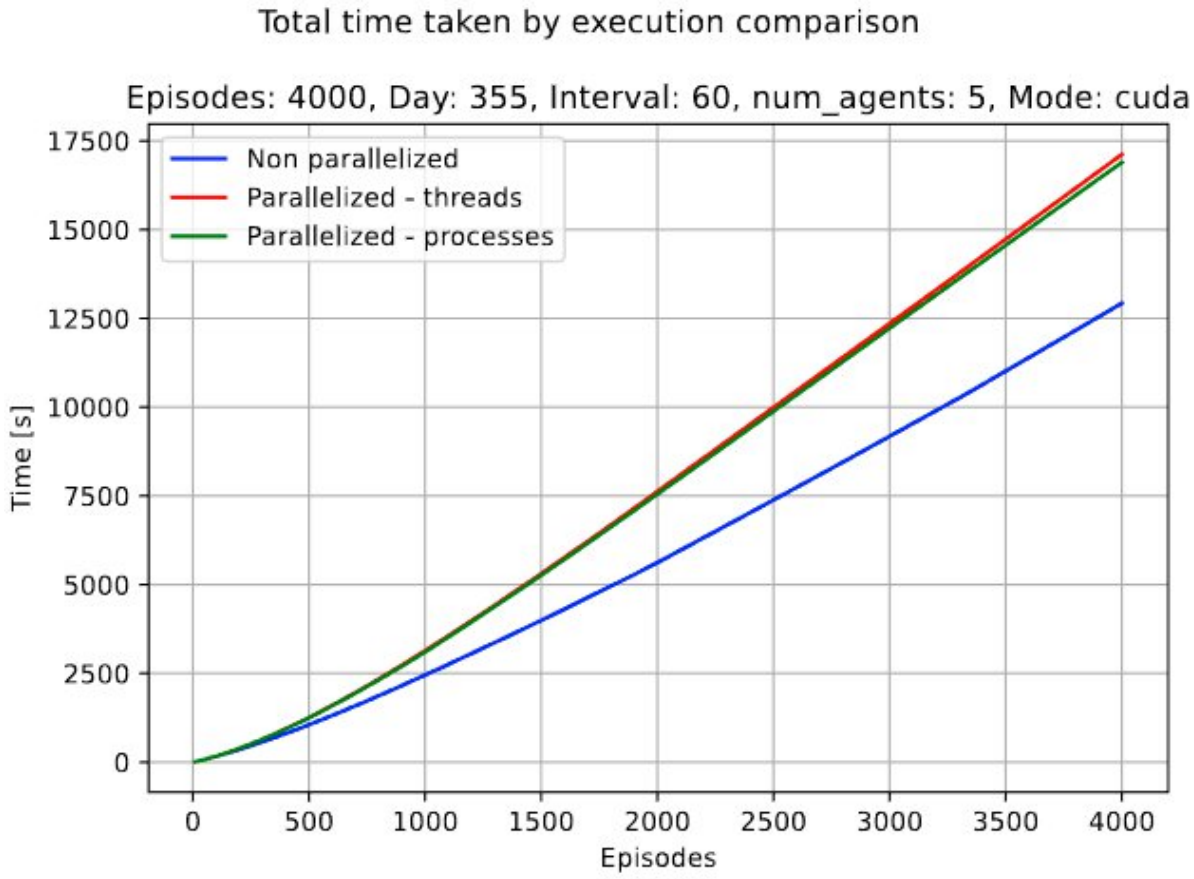


Adoption of GPU

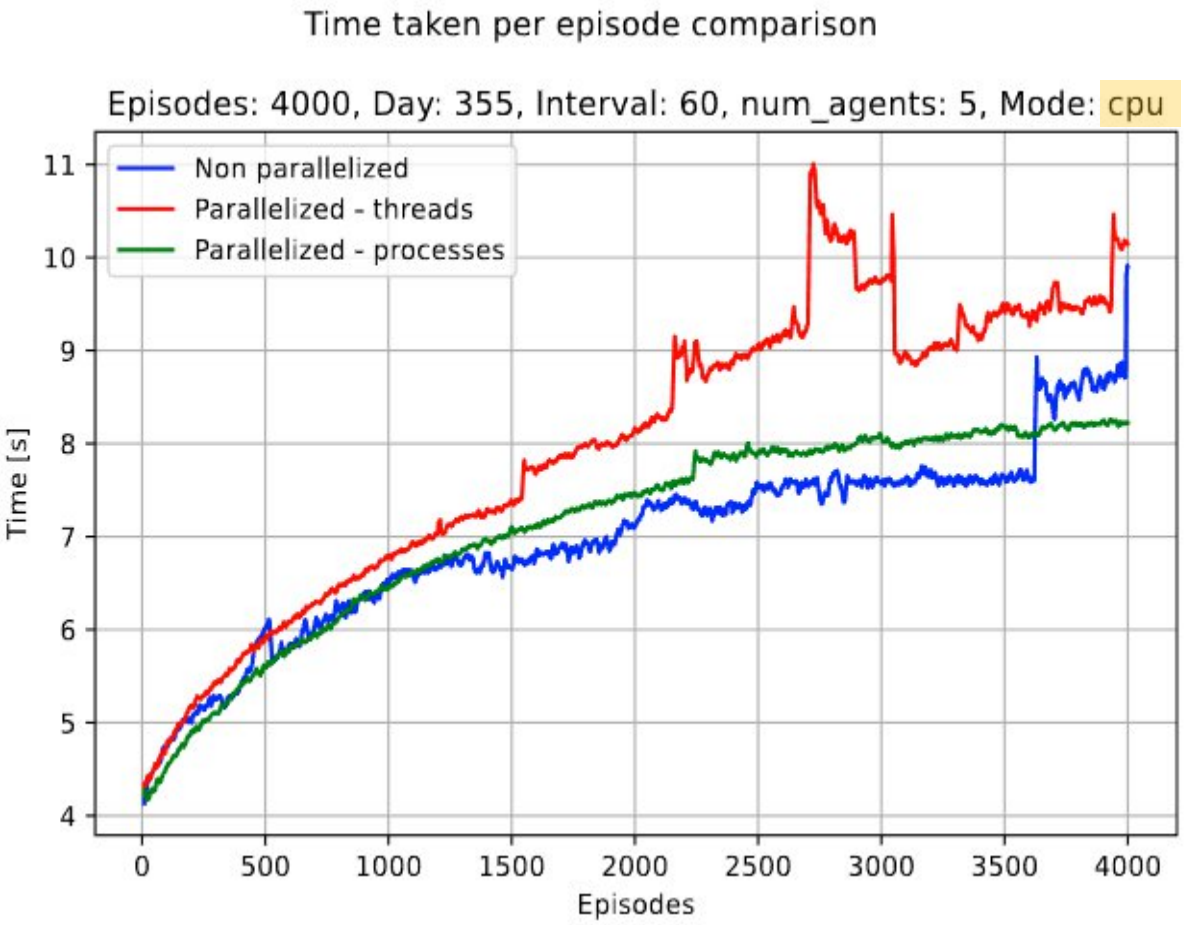
- Taking advantage of NVIDIA GPU in order to speedup calculations
- Up to # gpu cores for batch size for each agent before saturation
- Exploitation of *Cuda* for parallelizing operations of training
- Note: it is needed a downgrade of some libraries due to Pascal architecture (sm_61) of MX330 and GTX1050

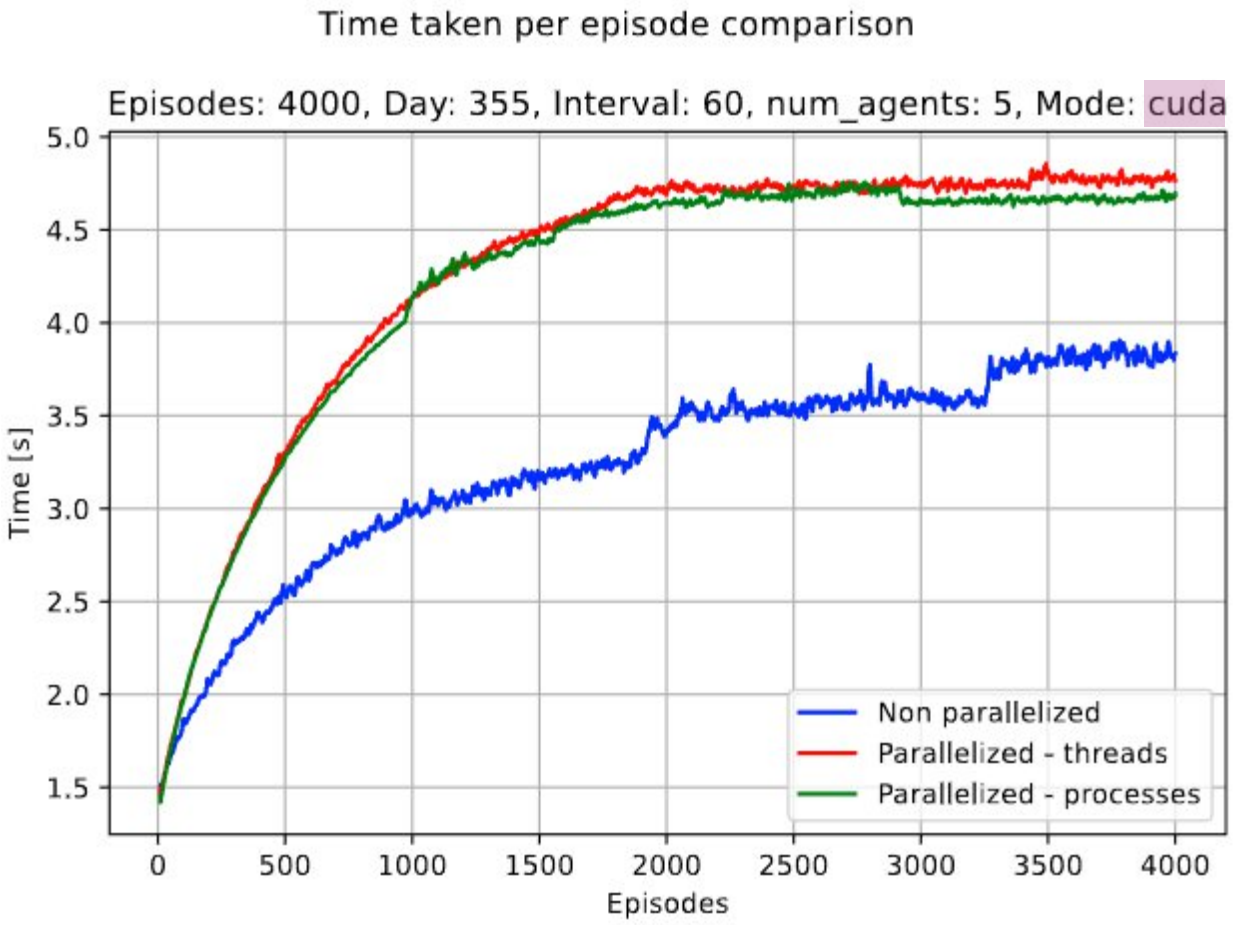
Time comparison





CPU vs GPU



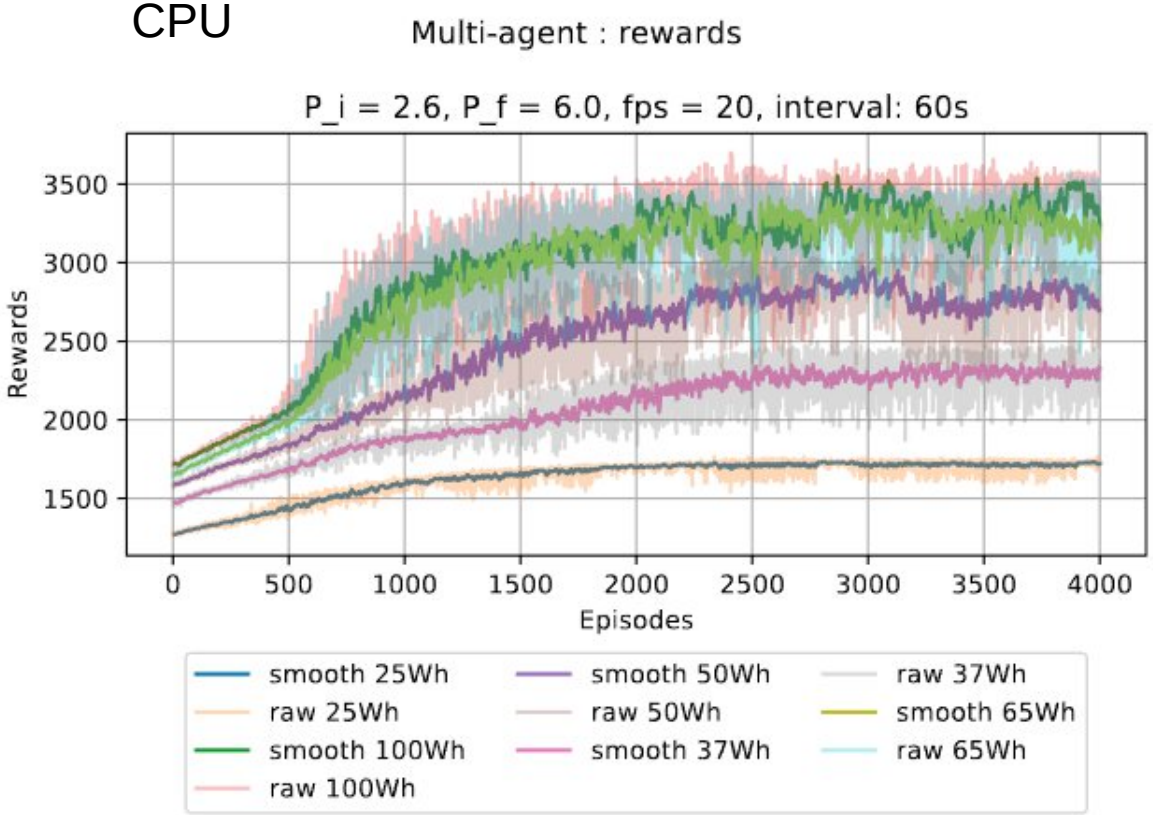


Serialized action decision and GPU usage “outsmarts” other combinations

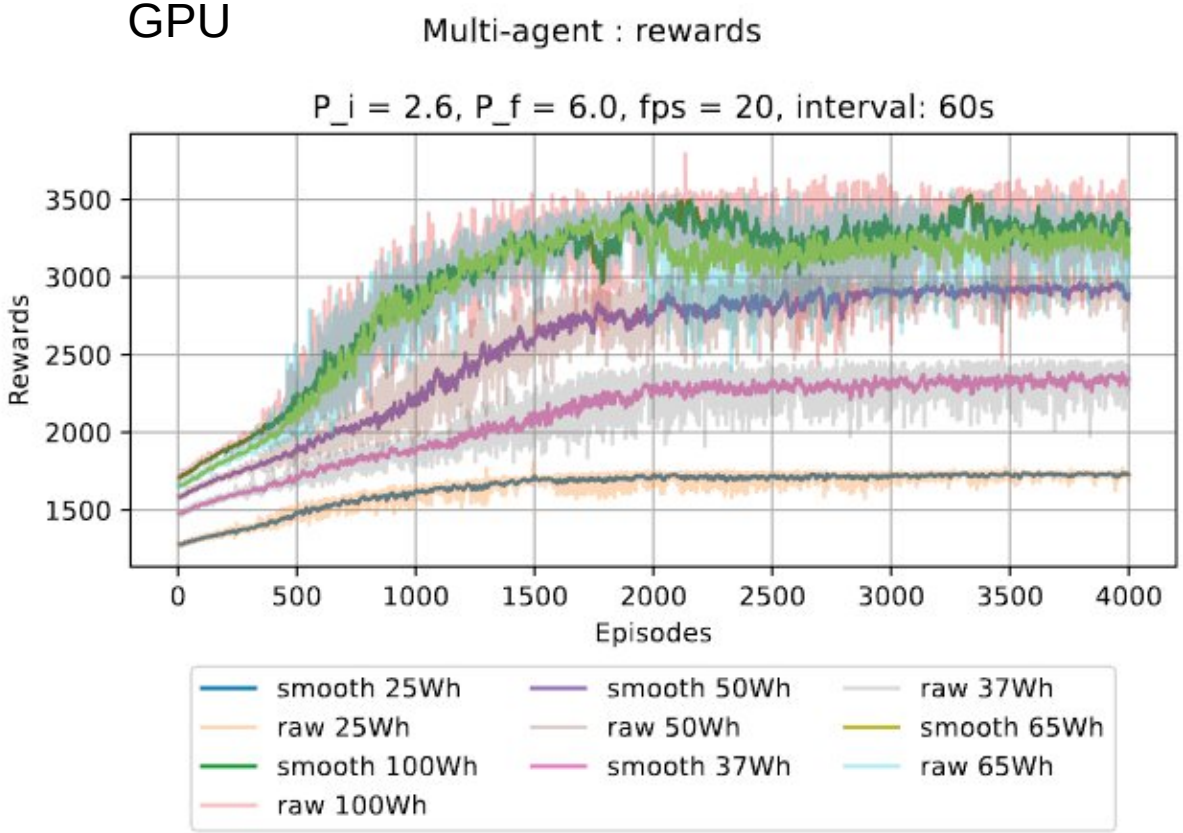
Other metrics comparison

- Using GPU instead of CPU leads to no differences in terms of the usual metrics tracked during training (ex. rewards):

CPU



GPU



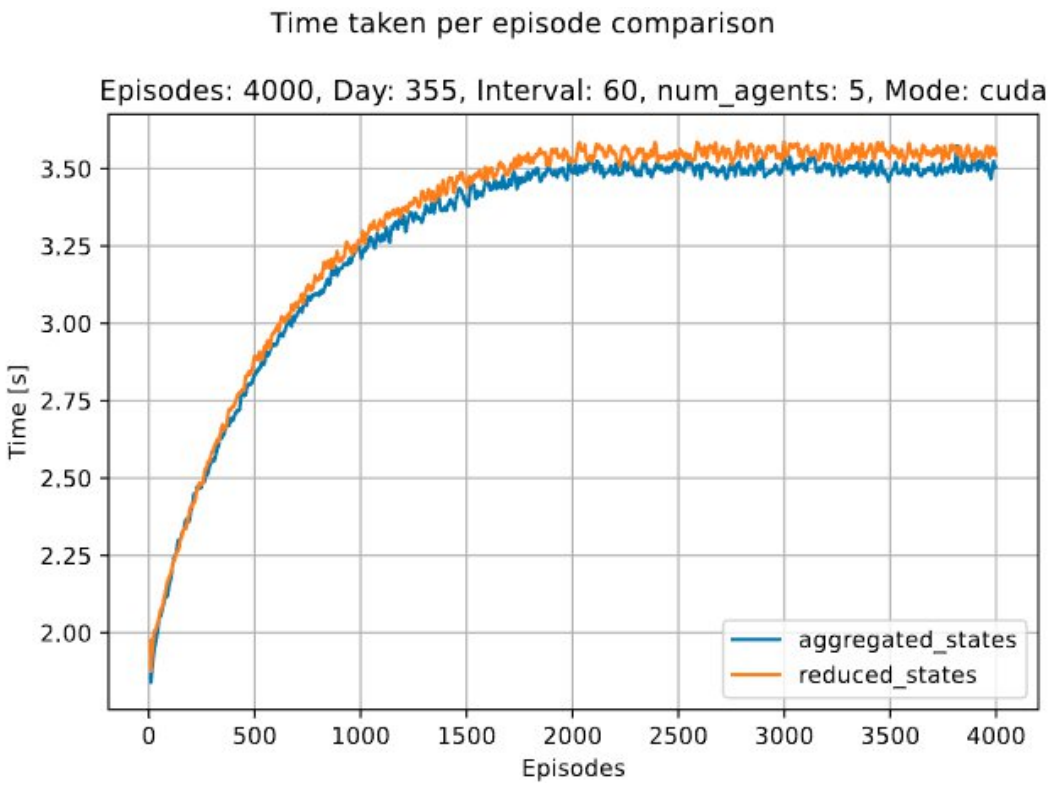
Information remotion from observation

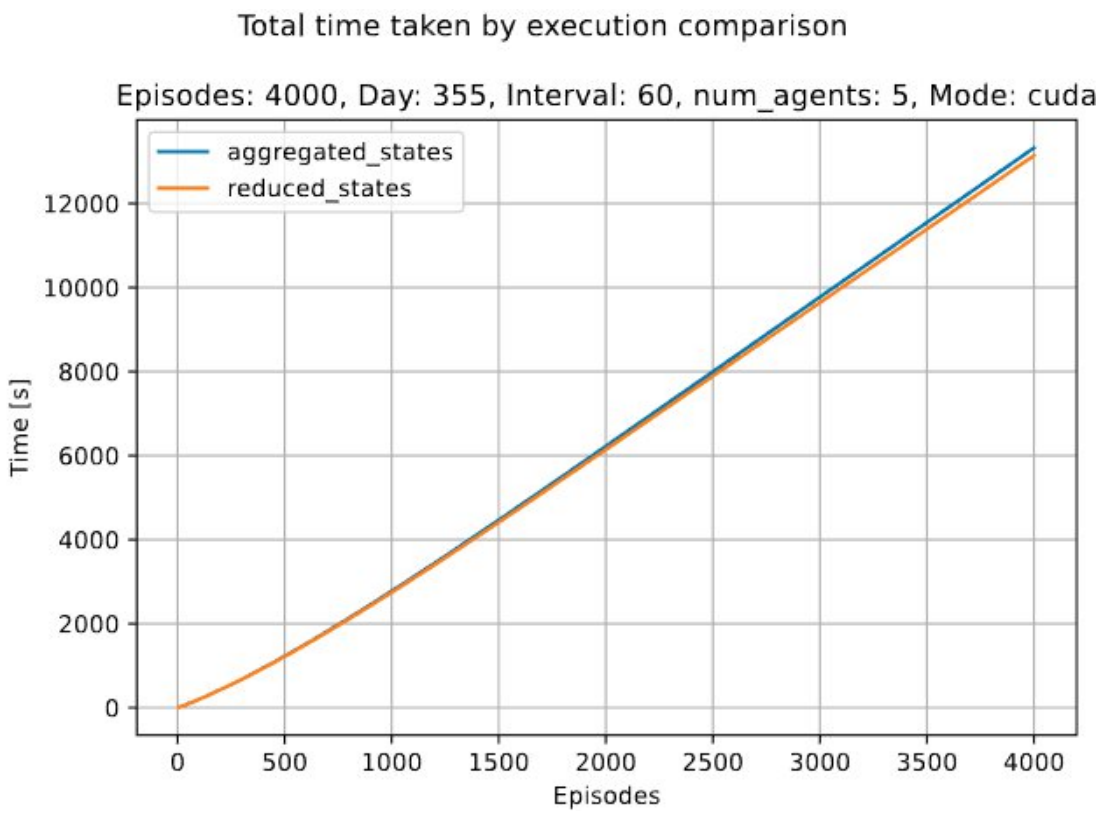
Lorenzo Pecorari - 1885161

Main goal

- Removing some information from observation in order to check behavior of agents with weaker knowledge about the system
- From $s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i]$
- To $s_t^i = [b_t^i, q_t^i, \tau_t^i, \mu b_t^i, \mu q_t^i]$
- Only mean battery level and mean storage level are available
- During tests, this attempt will be called “Reduced states”
- Tests done with DQN, day 355 and 5 agents

Time comparison

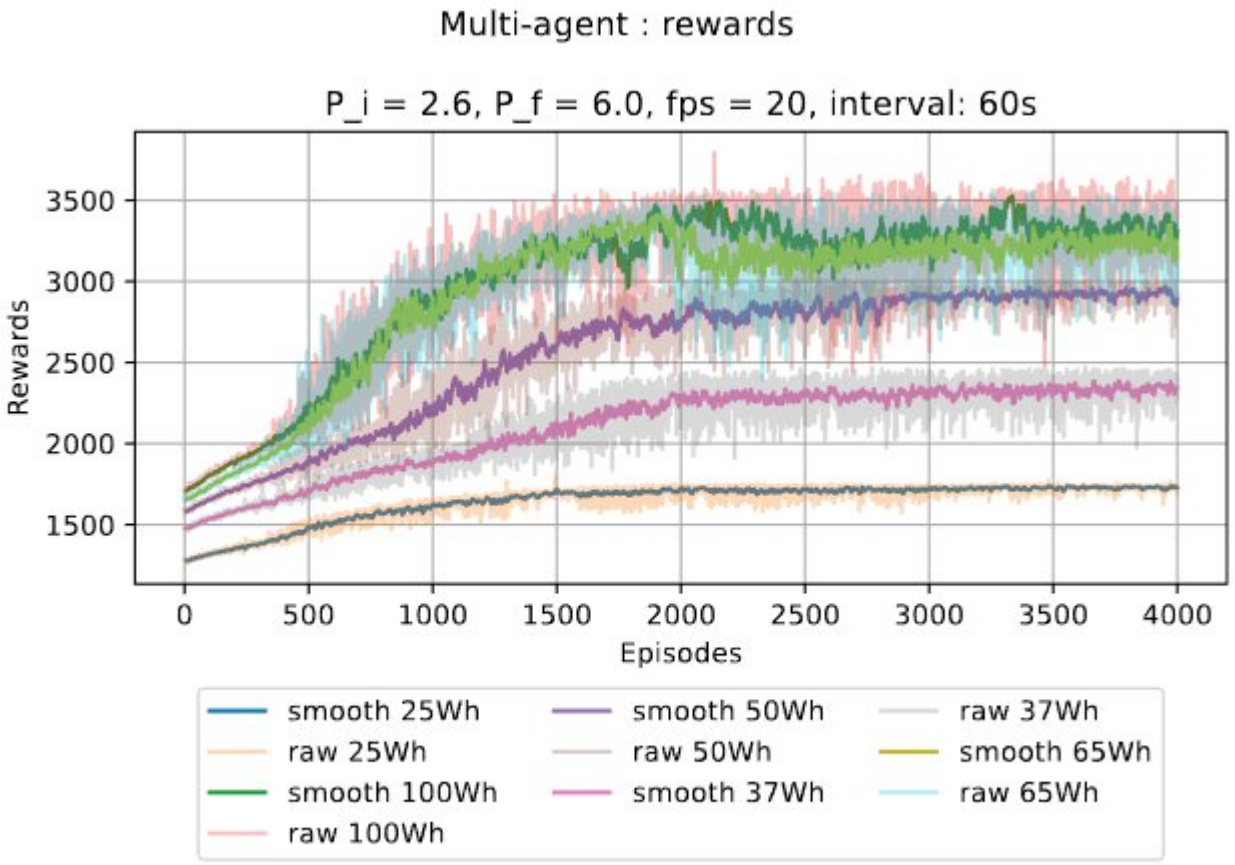




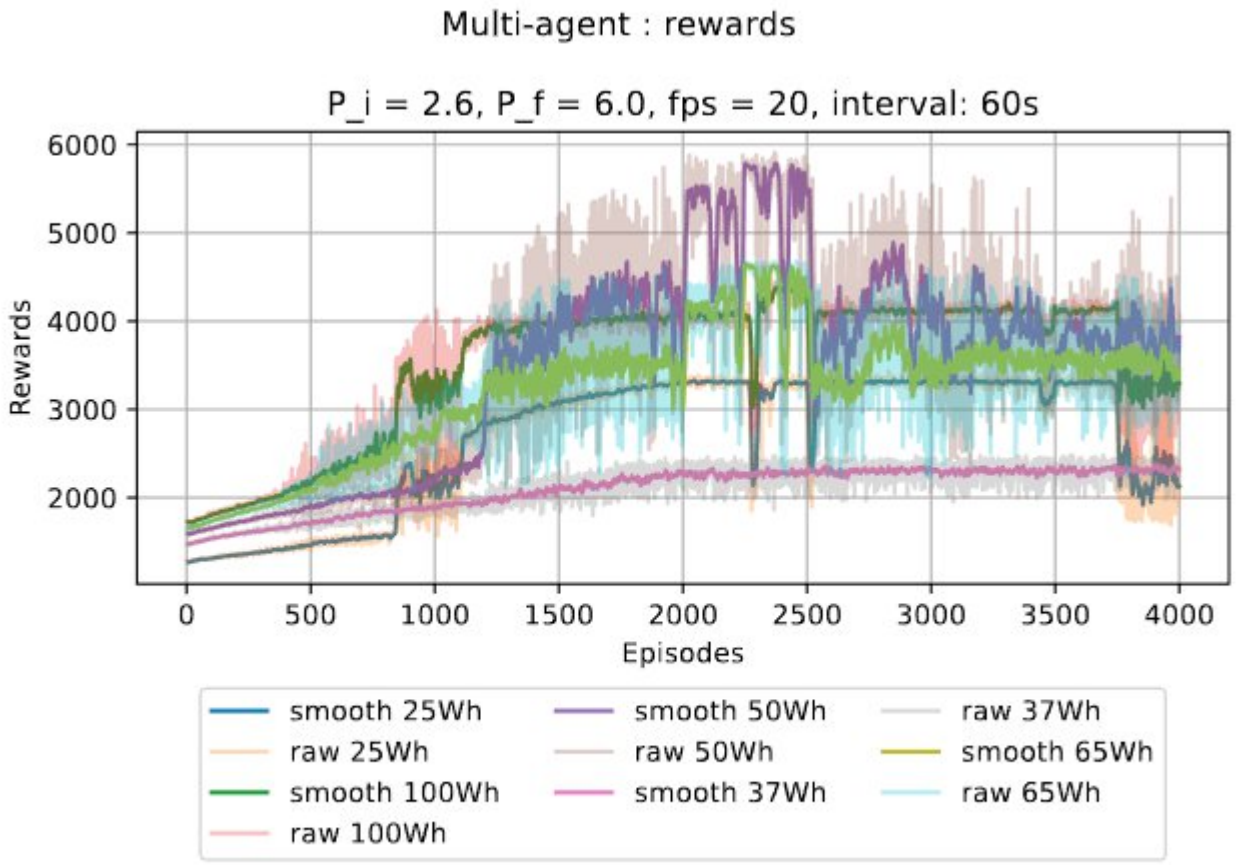
- Slightly higher required time for formulation with less information

Rewards - detail

BEFORE

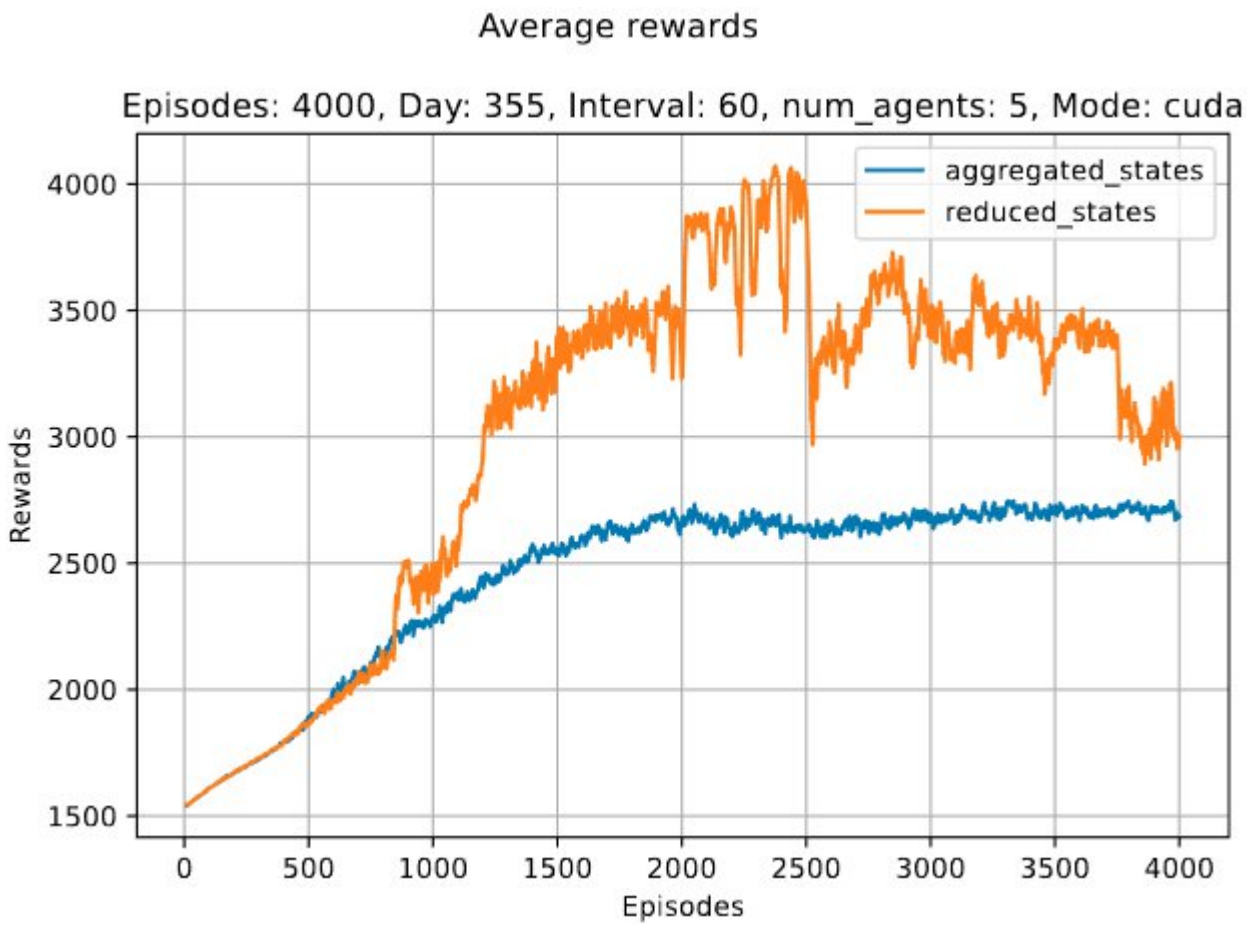


AFTER



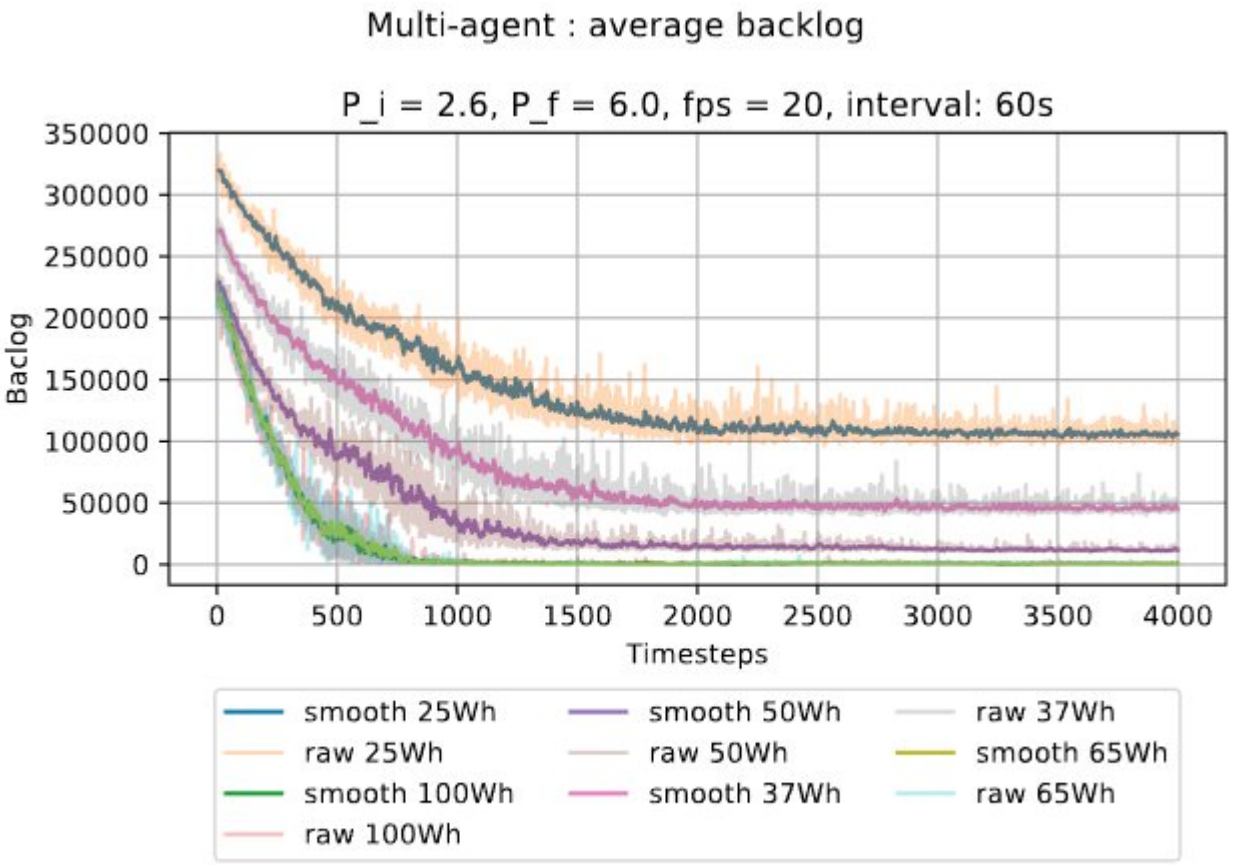
Rewards - average

- Average of rewards of 5 agents for each episode
- Less stable rewards for configuration with reduced information

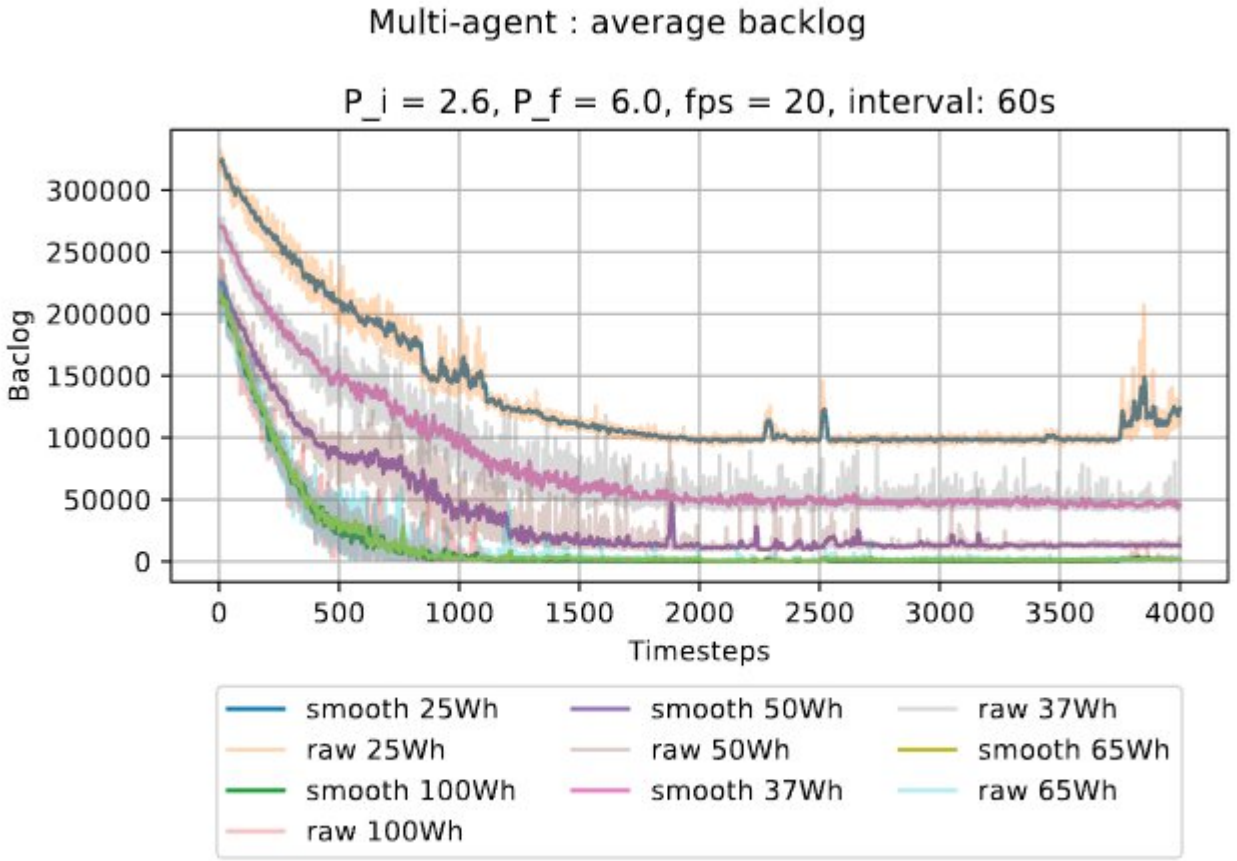


Backlogs

BEFORE

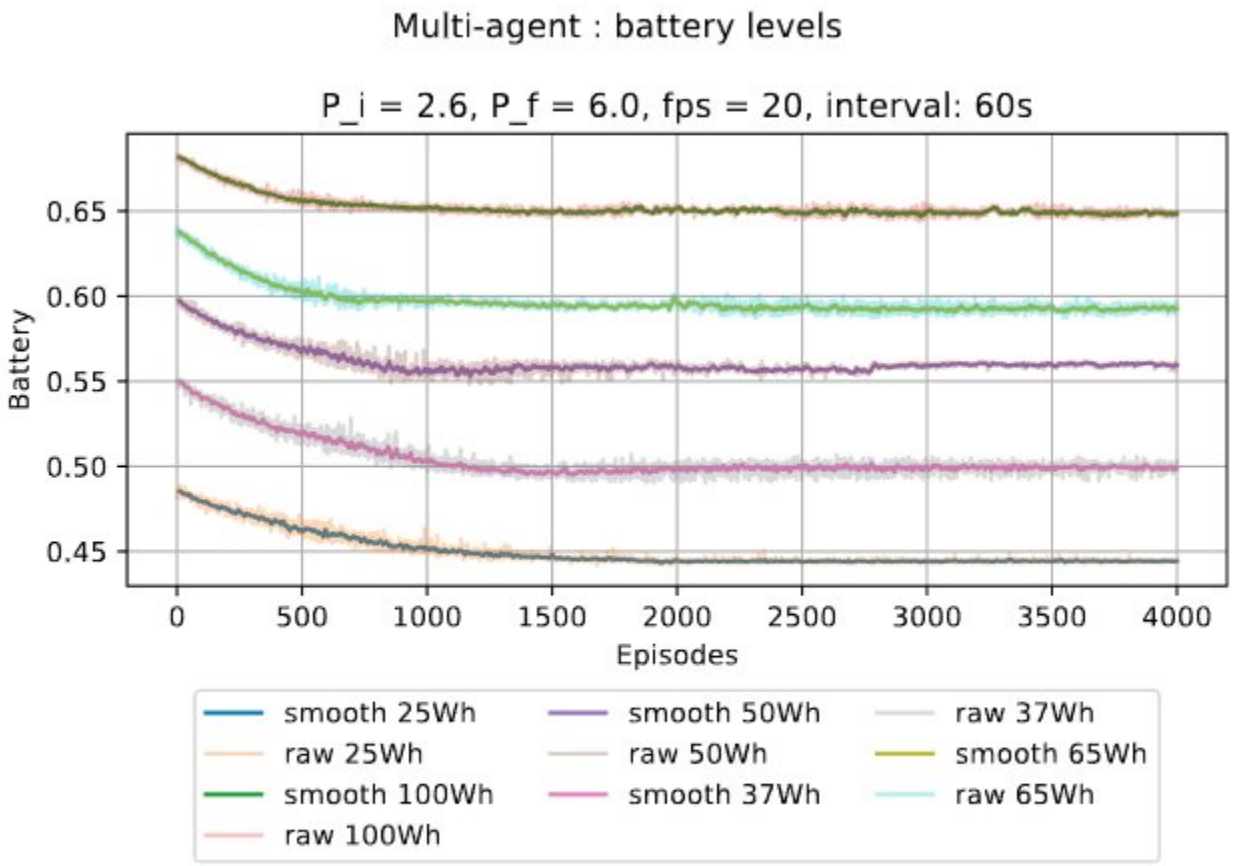


AFTER

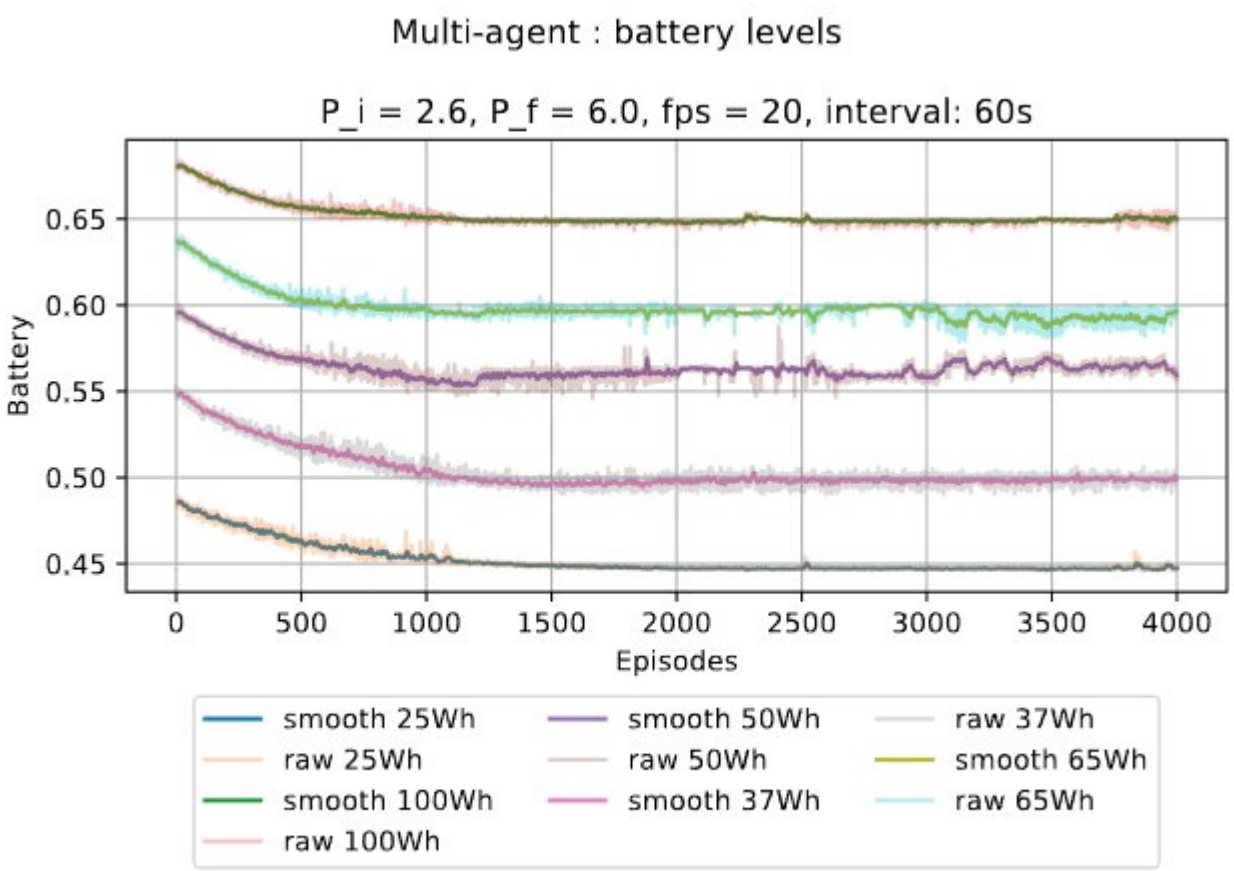


Batteries

BEFORE

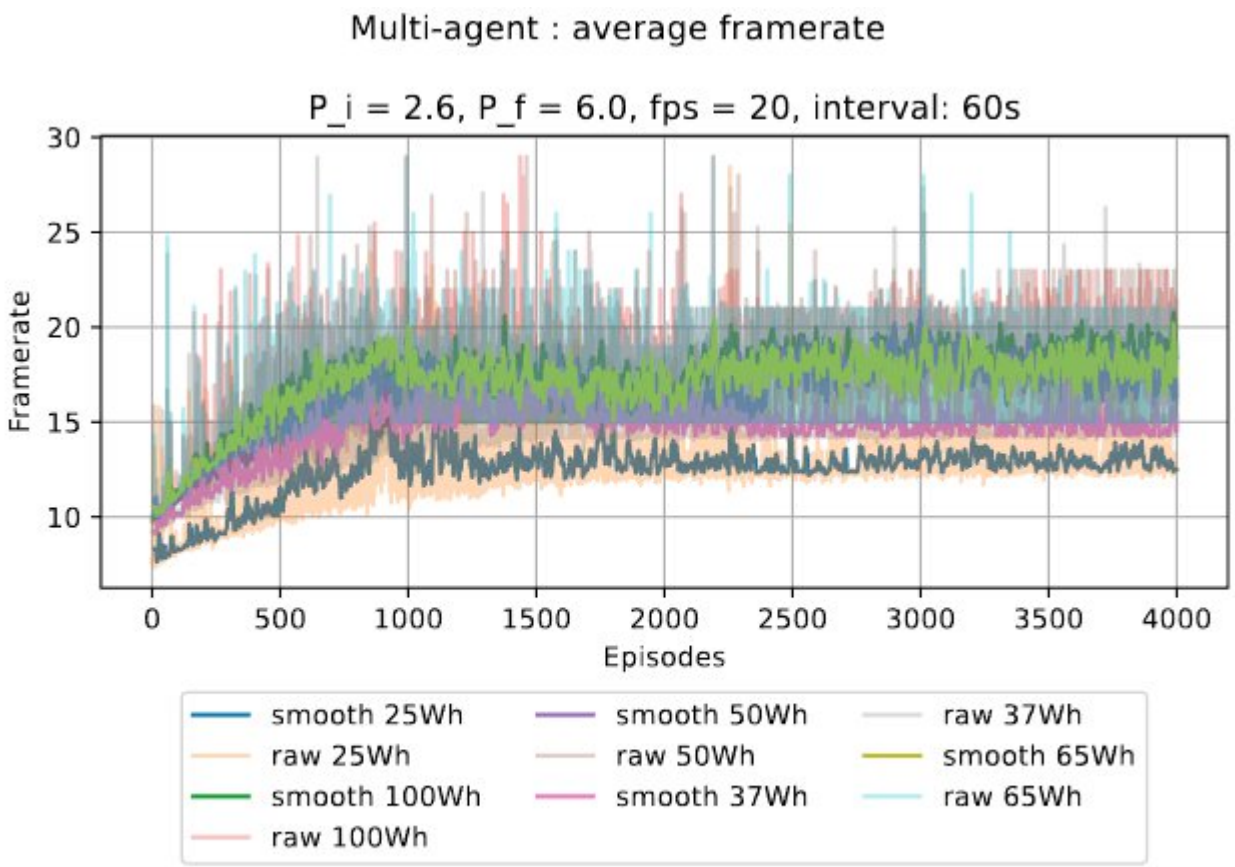


AFTER

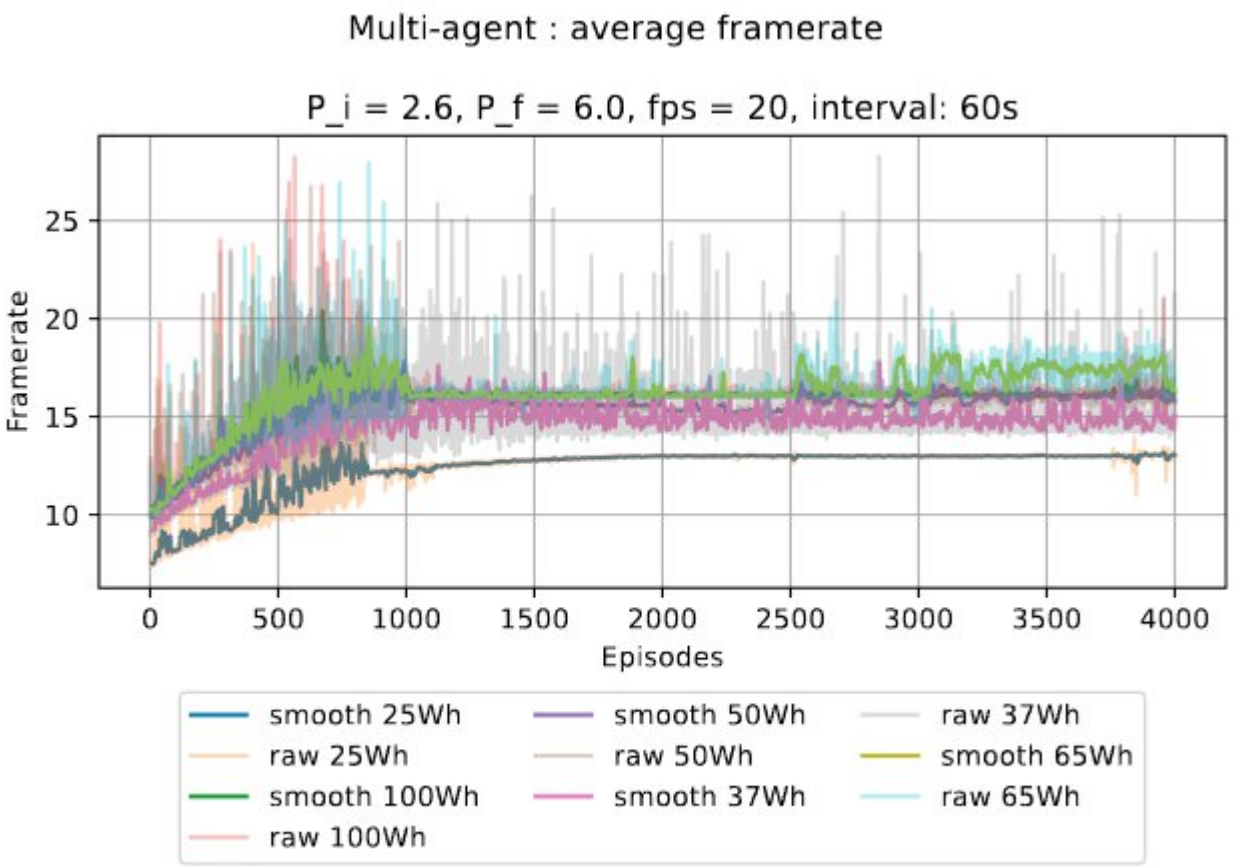


Framerates

BEFORE



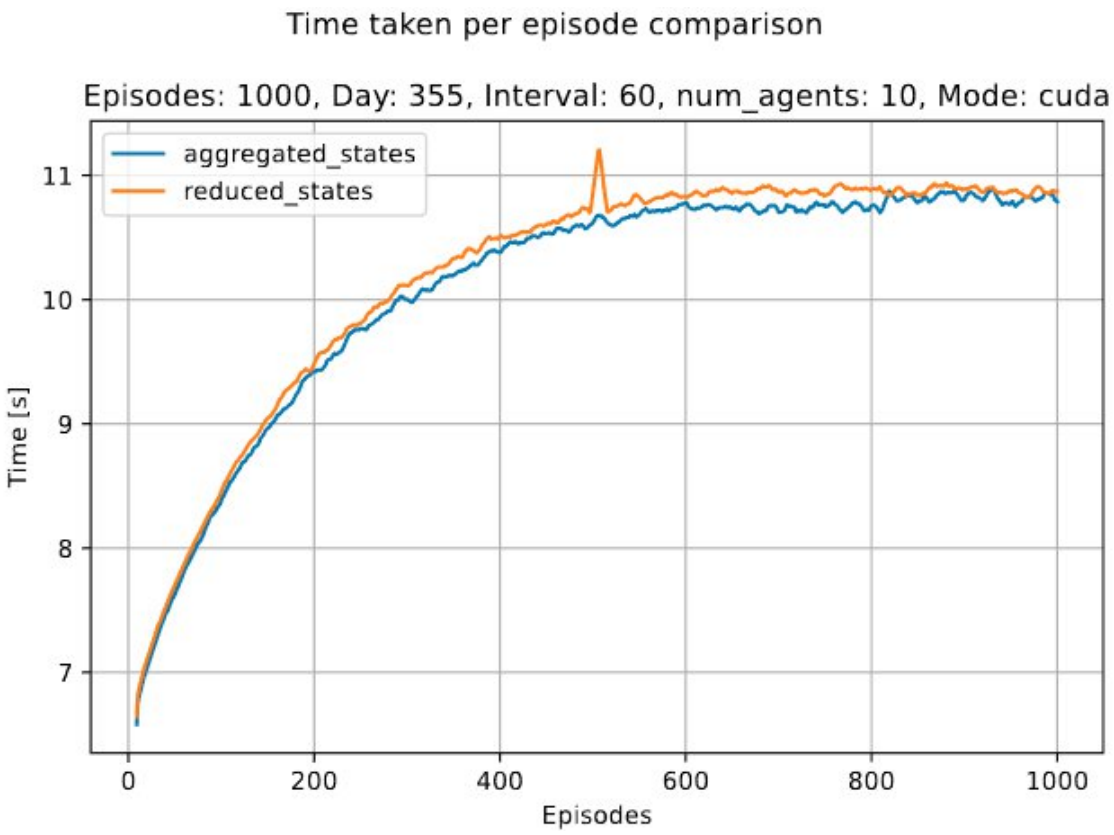
AFTER

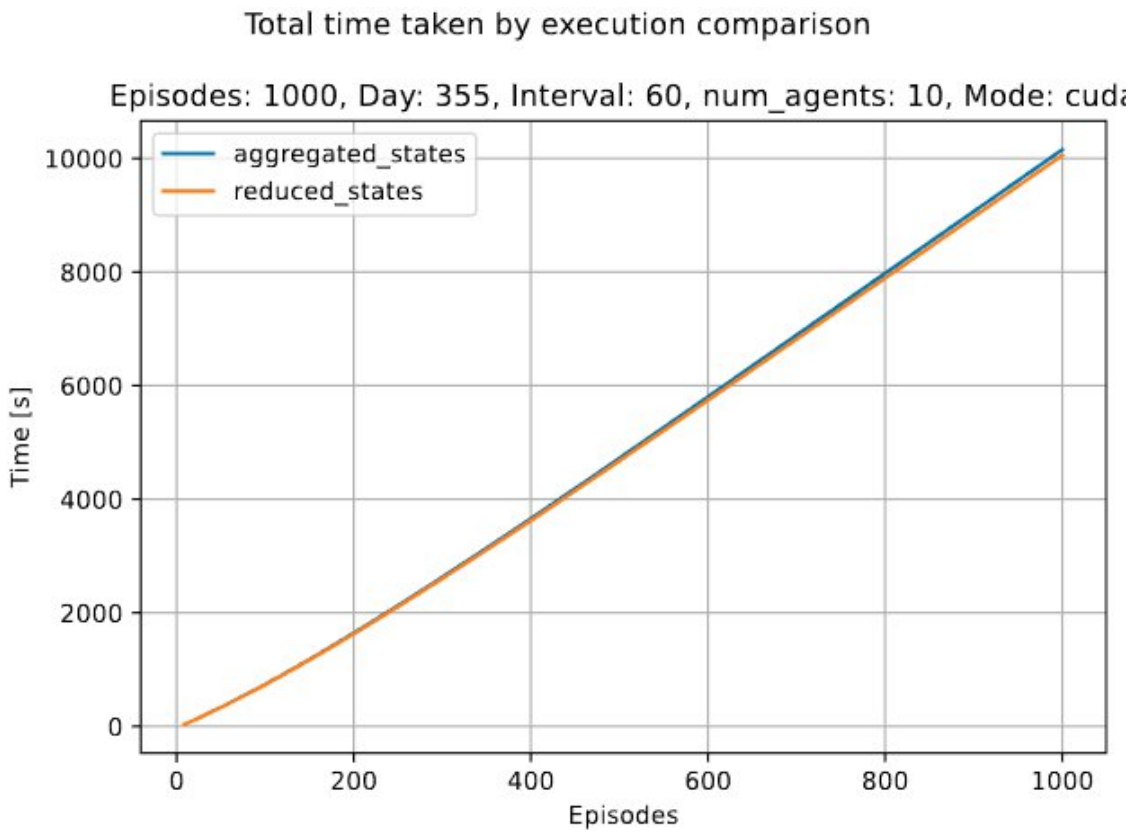


Extra: test with 10 agents

- Attempt to reproduce the previous tests with double the number of agents
- Run with 1000 episodes and ~50% of them used for exploration
- Less episodes for each run due to long time required (~3 h)
- Battery capacities: [25, 100, 50, 37, 65, 80, 40, 75, 55, 90] Wh
- Panel surfaces: [1.00, 0.50, 0.75, 0.85, 0.65, 0.55, 0.90, 0.30, 0.80, 0.52] m^2
- Usual solar data from Solcast, 2024

Time comparison

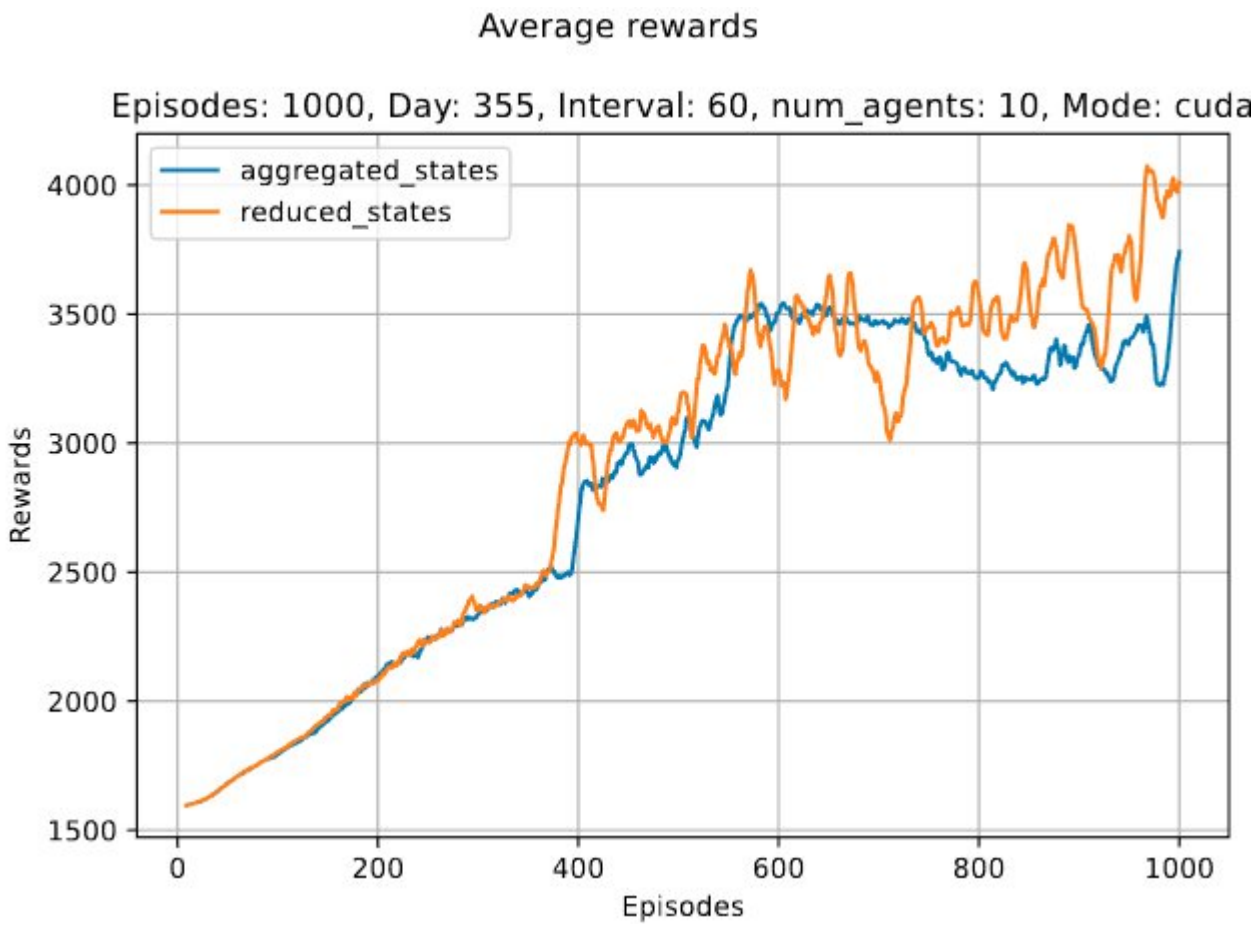




- Very similar results wrt less agents in the system

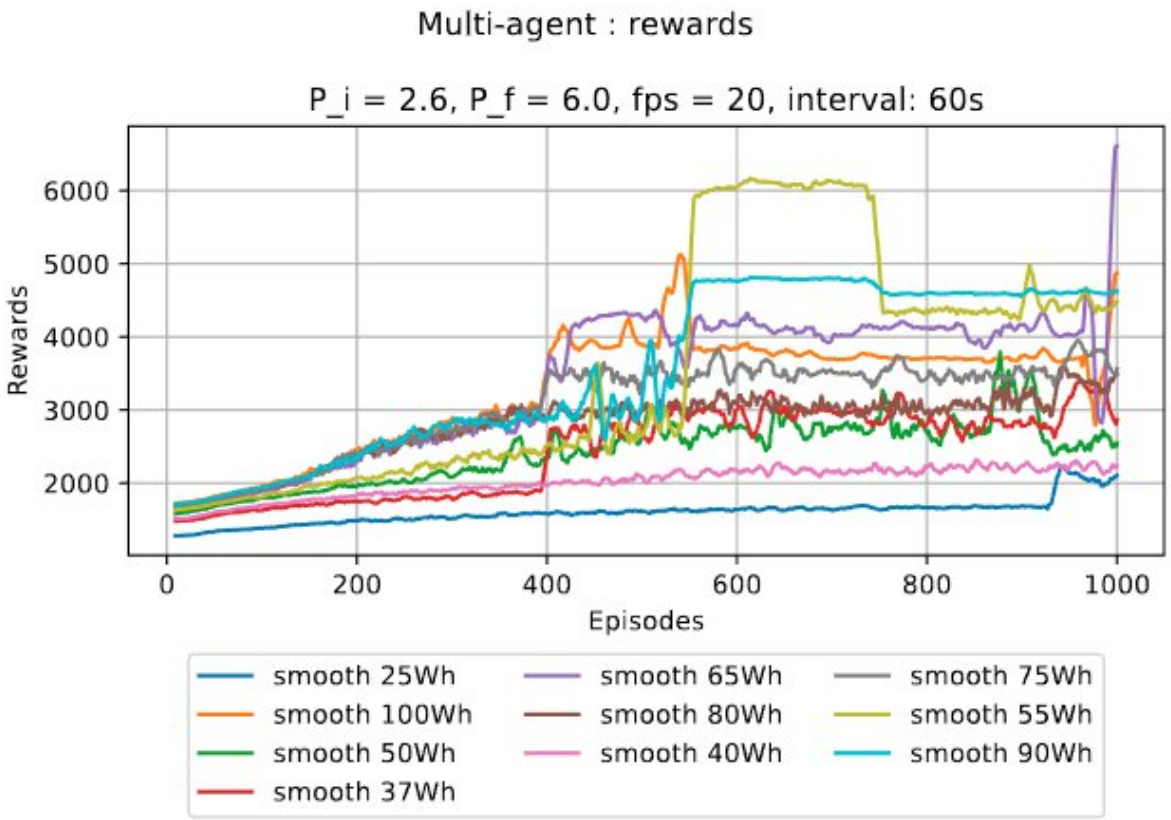
Rewards - average

- Similar trend as previous tests with less nodes
- Higher average rewards for initial formulation but reduced one always better

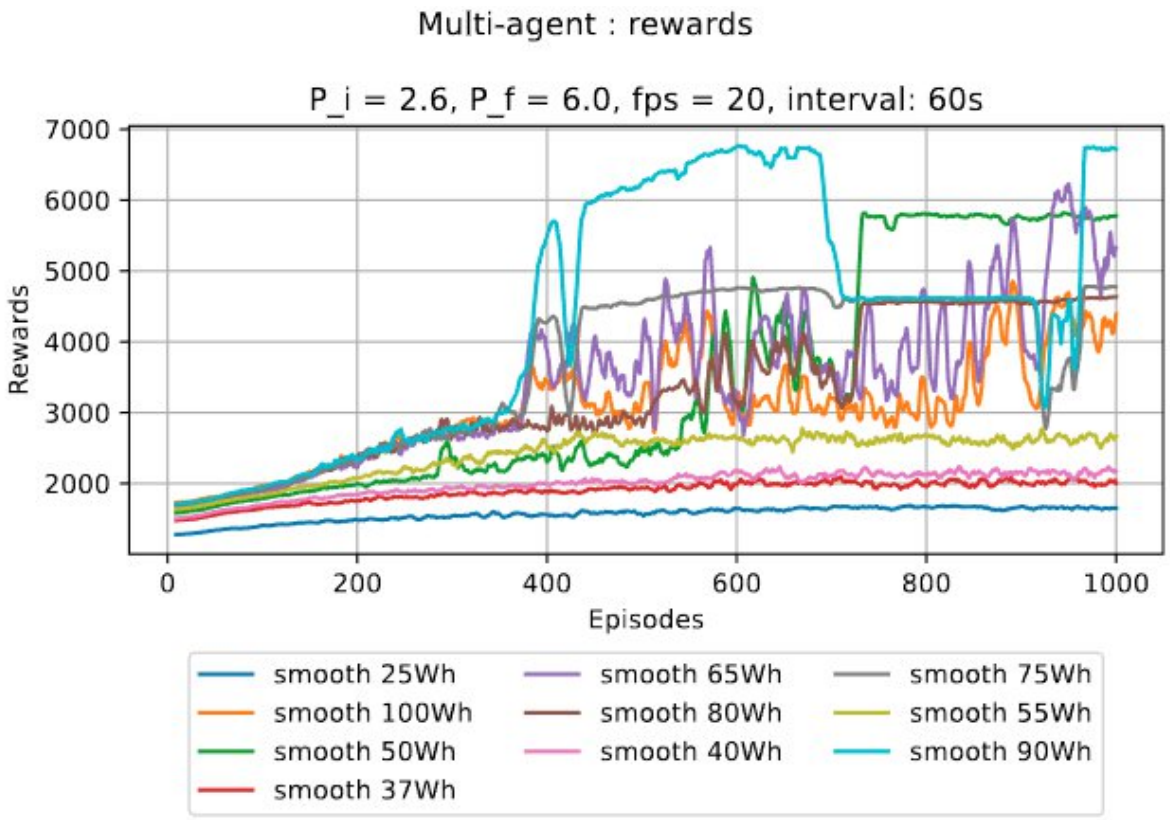


Rewards - detail

BEFORE

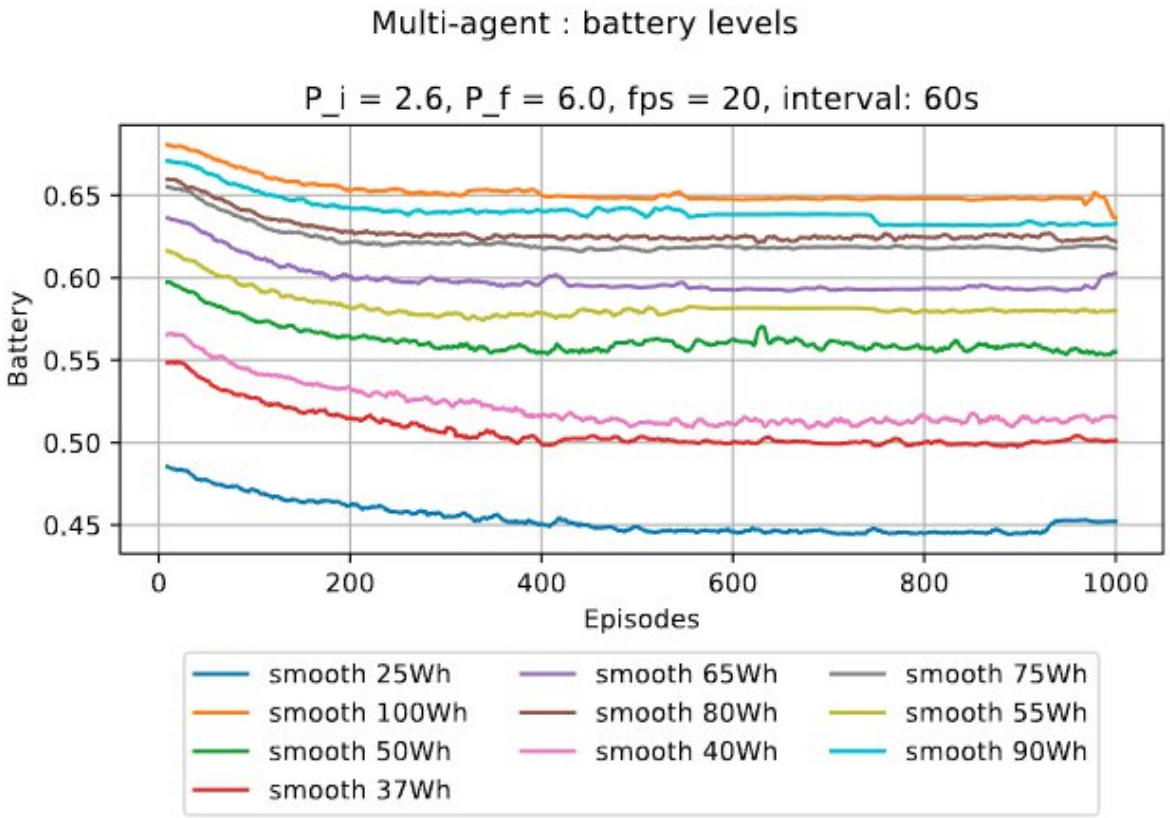


AFTER

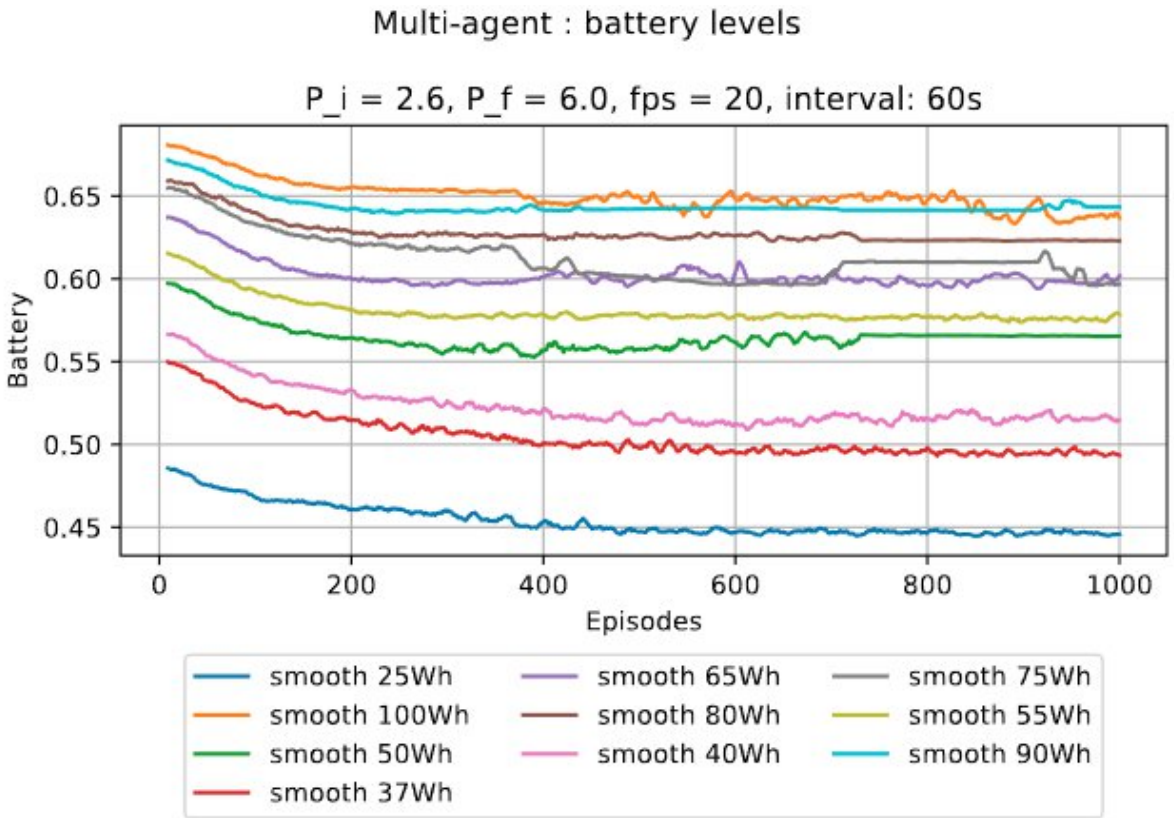


Batteries

BEFORE

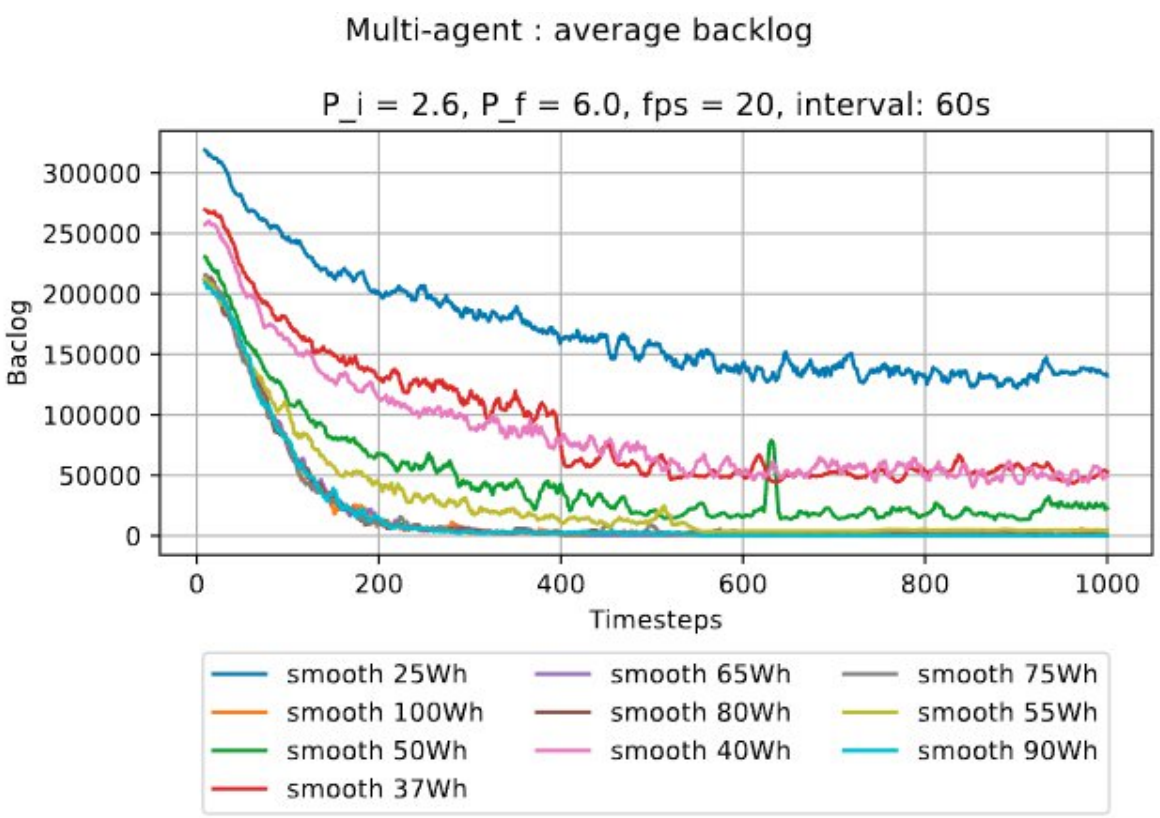


AFTER

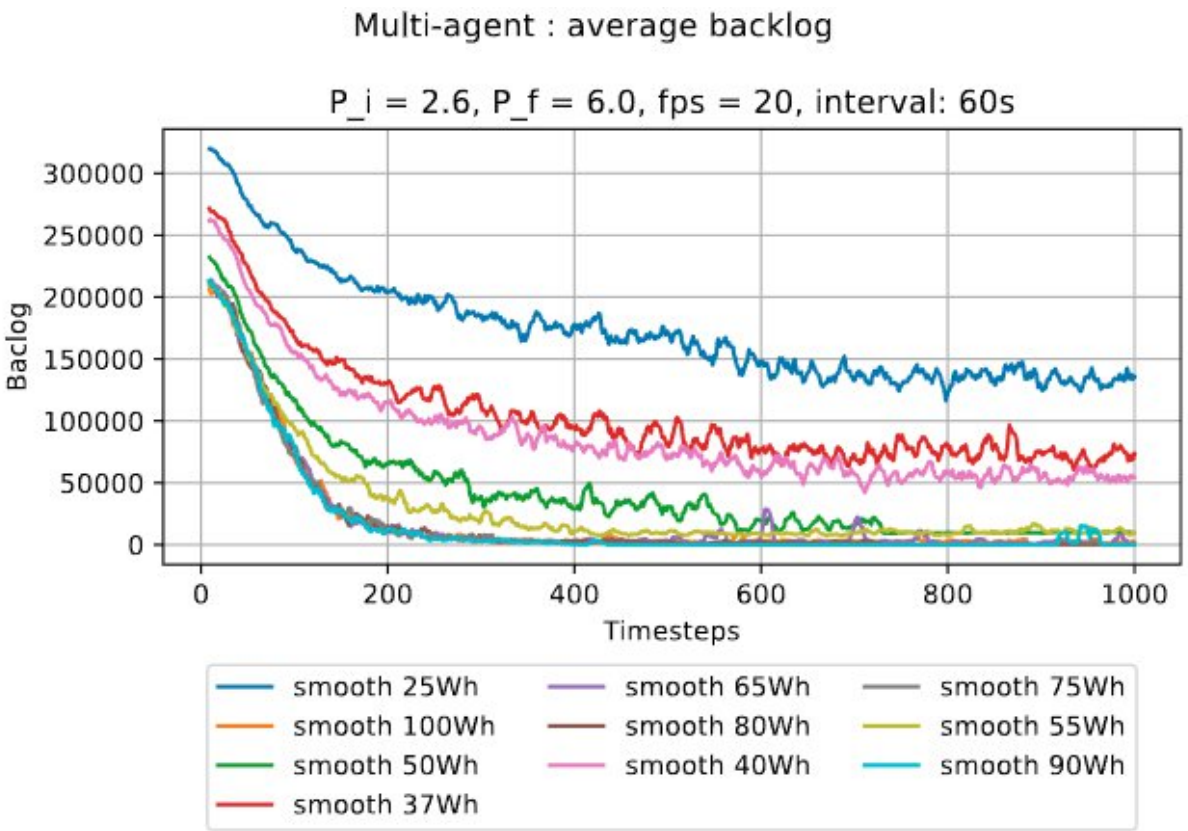


Backlogs

BEFORE

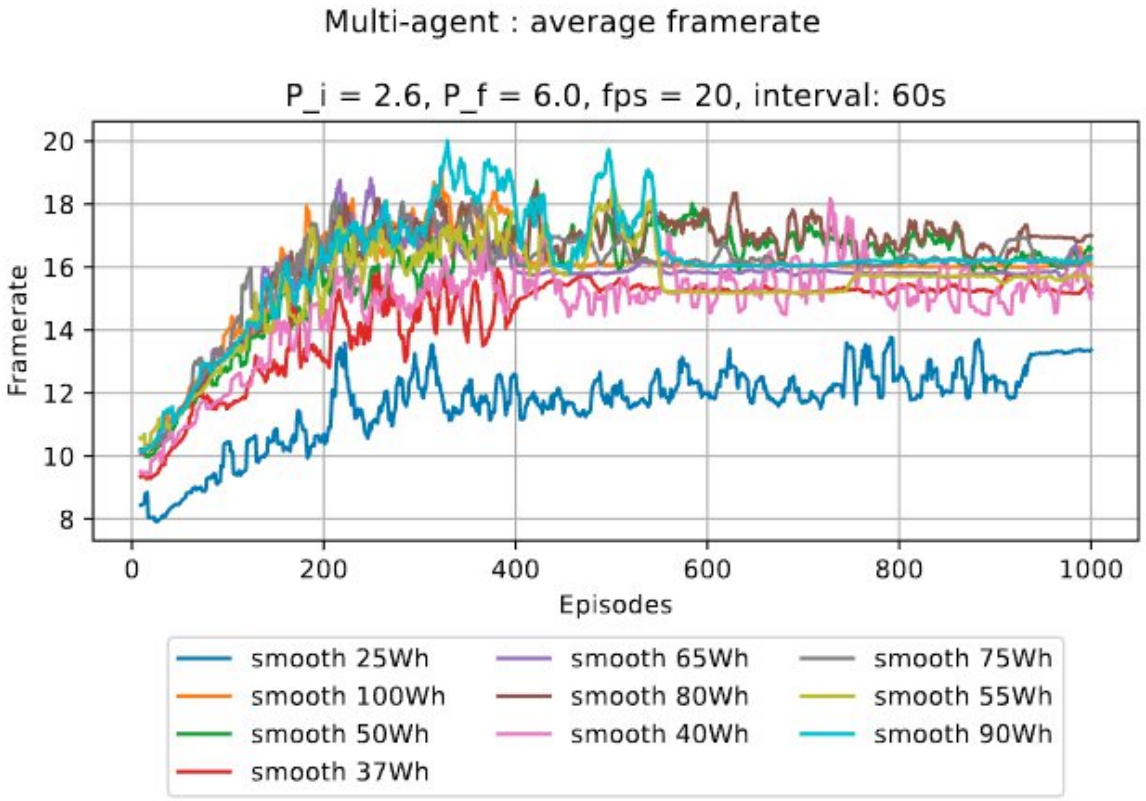


AFTER

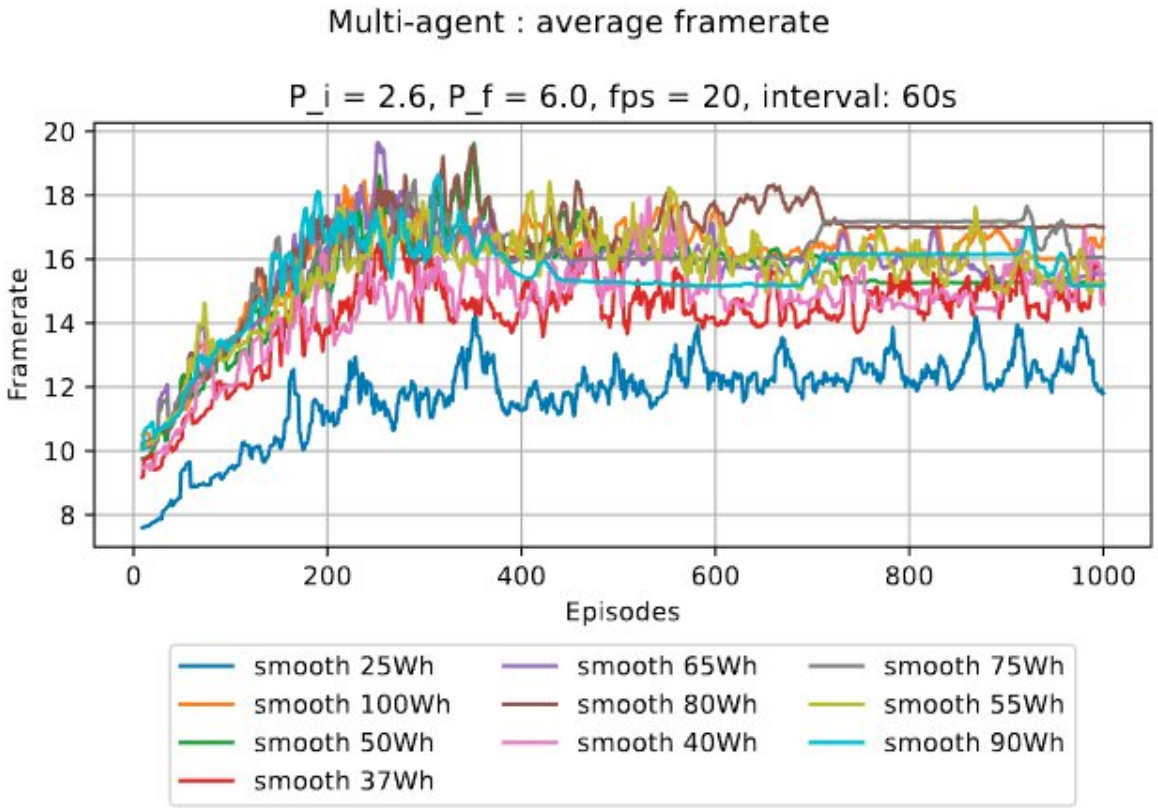


Framerates

BEFORE



AFTER



Explicit matching

Lorenzo Pecorari - 1885161

Formulation

- Adding a binary variable in observation for informing agents if offloading matching happened at time t-1
- From $s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i]$
- To $s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i, match_t^i]$
- Or to $s_t^i = [b_t^i, q_t^i, \tau_t^i, \mu b_t^i, \mu q_t^i, match_t^i]$ (“reduced states”)
- Increasing of 1 dimension the observation, agents can know if offloading at t-1 was successful ($match_t^i = 1$) or not ($match_t^i = 0$)

More expressive state aggregation

Lorenzo Pecorari - 1885161

From average to more expressive metric

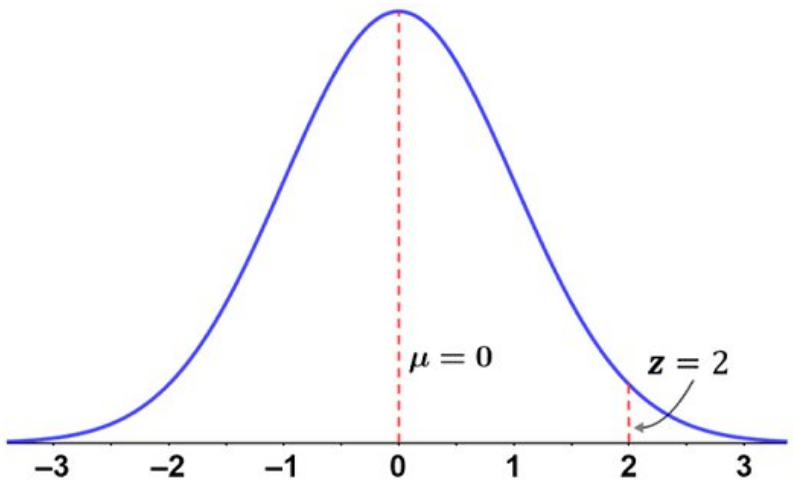
- Minimum, maximum and average allow agent to know their state wrt the others
- Reducing to only average (of others) may lead to loss of information about the range
- Using z-score might help to define the “position” of each agent compared to others
- However, it is unbounded and it may be needed a clipping

Z-Score

- z : $\mathbb{R}^k \rightarrow \mathbb{R}$
- z(x) = $\frac{x-\mu}{\sigma}$, where:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$$

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$

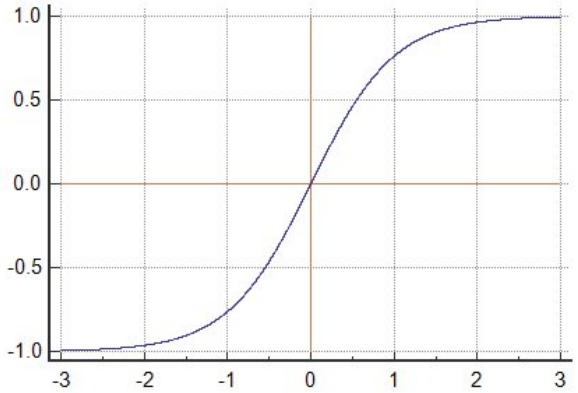


- “how many std deviation x is far from the mean” or “how far x is from the group”
- Applying this to battery and backlog level for each agent, it allows to know which is the situation of the node without using three variables

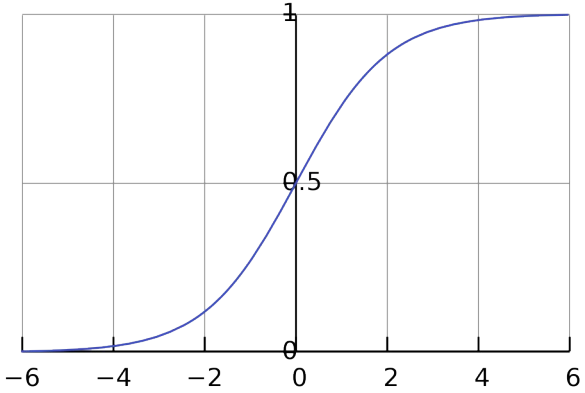
Limiting the z-score

- Due to the range (-∞, +∞) of the output of z(x), it can be applied a limitation of its value to smaller intervals
- Ex: hyperbolic tangent function and sigmoid can cope such problem

- tanh: $\mathbb{R}^k \rightarrow [-1, +1]$
- $\tanh(x) = \dfrac{e^{2x} - 1}{e^{2x} + 1}$



- sigmoid: $\mathbb{R}^k \rightarrow [0, +1]$
- $\text{sigmoid}(x) = \dfrac{1}{1 + e^{-x}}$



Formulation with hyperbolic tangent

- Using z-score for finding position of agent metrics wrt the group
- Using tanh for delimit the z-score
- From $s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i]$
- To $s_t^i = [b_t^i, q_t^i, \tau_t^i, tz(b_t^i), tz(q_t^i)]$

from 9 dimensions to 5

- where:
  - $tz(x) = \tanh(z(x))$,
  - $\tanh(z(x)) = \frac{e^{2z(x)} - 1}{e^{2z(x)} + 1}$
  - $z(x) = \frac{1}{1 + e^{-x}}$

Formulation with hyperbolic tangent

- It can be preferred to use tanh instead of sigmoid since, when the node has same metrics values as the others, with sigmoid the variable becomes 0.5 but with tanh 0 (centrality)
- It might be possible to extend this formulation with the binary variable which tells to the agent if the offloading successfully happened at t-1, so observation becomes:

$$s_t^i = [b_t^i, q_t^i, \tau_t^i, tz(b_t^i), tz(q_t^i), \mathrm{match}_t^i]$$